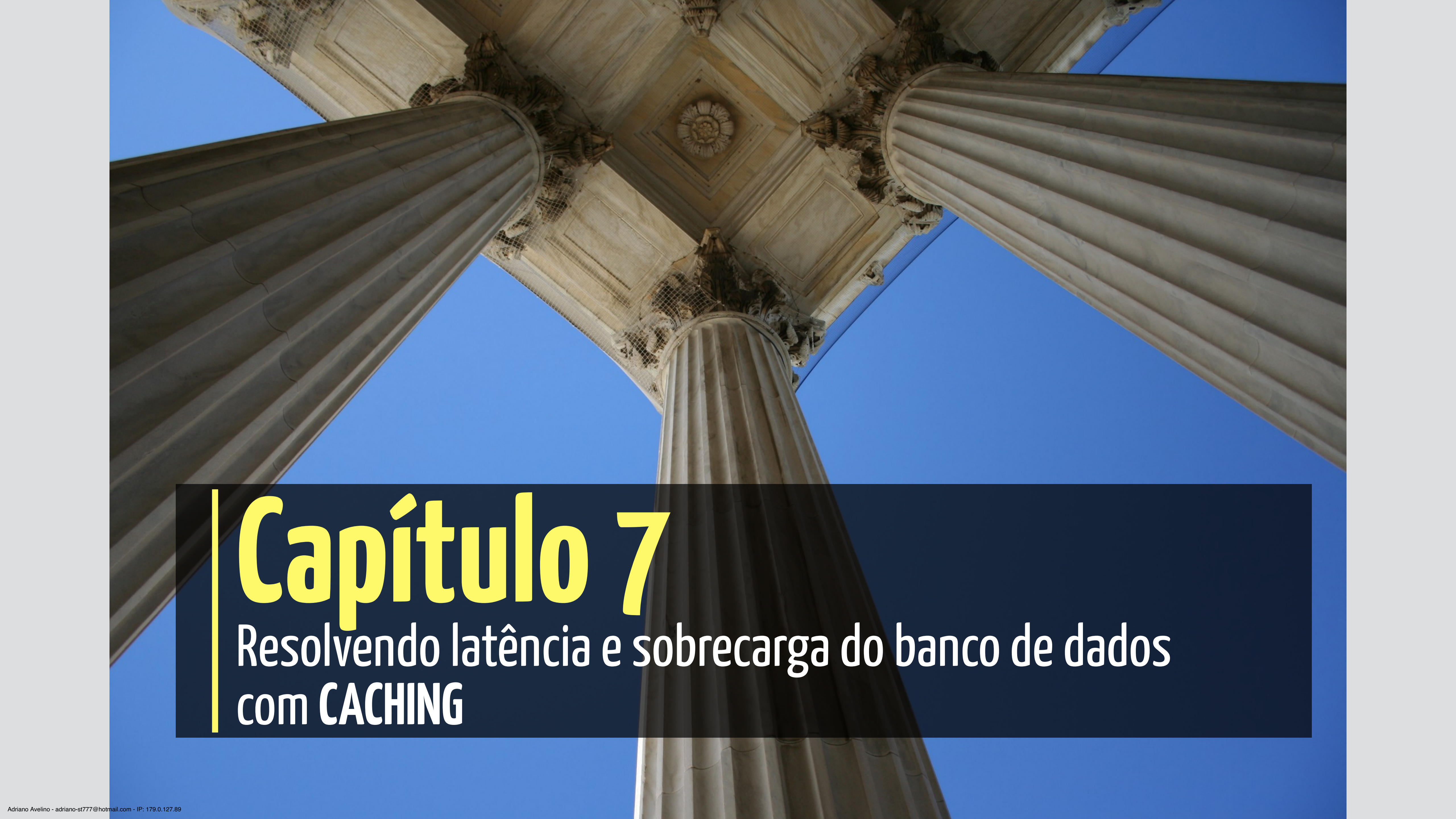


A low-angle, upward-looking photograph of several large, fluted classical columns. The columns are made of light-colored stone or marble and are set against a clear, bright blue sky. The perspective creates a sense of height and grandeur, with the columns converging towards the top of the frame. The top of the columns shows ornate capitals.

Capítulo 7

Resolvendo latência e sobrecarga do banco de dados
com CACHING

Mesmo com todas essas
melhorias...

■ HTTP Request (success)



...PICOS de acessos
podem aumentar
nosso **response time**

```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```



```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```

```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```

```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```



```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```



```

@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}

```

```
@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow(() -> new ResponseStatusException(NOT_FOUND, "Nota Fiscal não encontrada"));

        List<ItemDoReciboResponse> itens = nota.getItems().stream()
            .map(item -> new ItemDoReciboResponse(
                item.getDescricao(),
                item.getQuantidade(),
                item.getPrecoUnitario(),
                item.getPrecoUnitario().multiply(item.getQuantidade())
            ))
            .collect(Collectors.toList());

        BigDecimal total = itens.stream()
            .map(ItemDoReciboResponse::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        return new ReciboResponse(
            nota.getNumero(),
            nota.getDataDeEmissao(),
            nota.getCliente().getNome(),
            itens,
            total
        );
    }
}
```

CACHING

(aliviando o banco de dados)

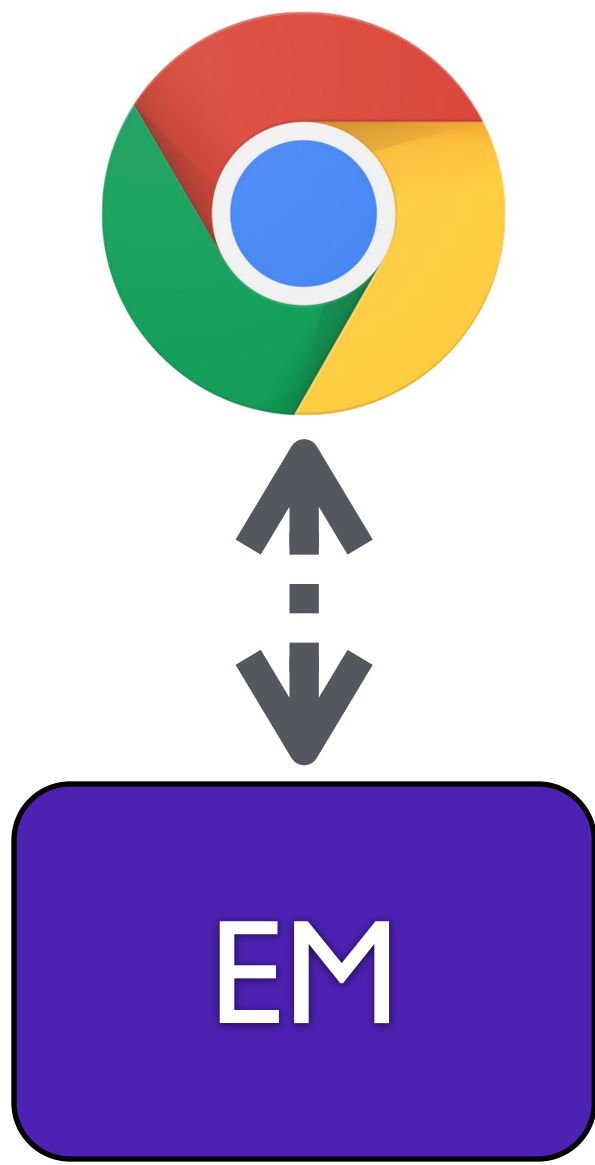
Carregando uma nota por ID

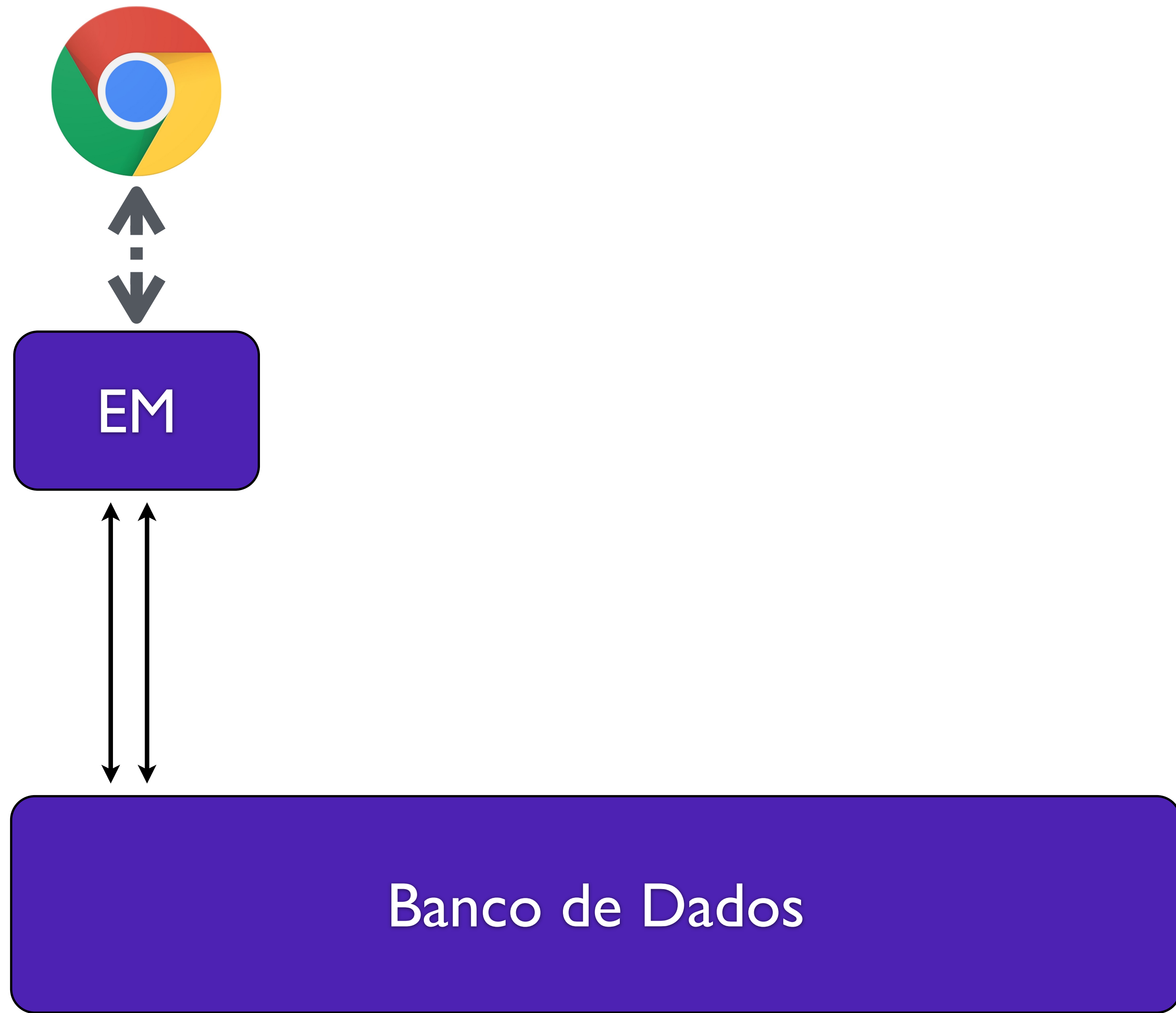
```
NotaFiscal nf = repository.findById(42);
```

Carregando uma nota por ID

```
NotaFiscal nf = entityManager  
                .find(NotaFiscal.class, 42);
```

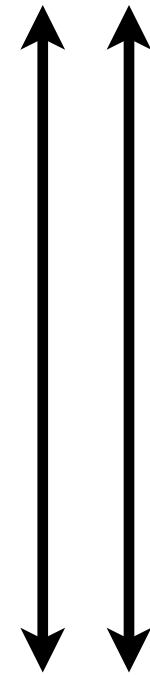








```
NotaFiscal nf = entityManager  
                .find(NotaFiscal.class, 42);
```



Banco de Dados

Hibernate executa I select

```
NotaFiscal nf = entityManager.find(NotaFiscal.class, 42);
```


Hibernate executa I select

```
NotaFiscal nf = entityManager.find(NotaFiscal.class, 42);
```

```
SELECT nf.*  
FROM NotaFiscal nf  
WHERE nf.id = 42
```

Carregando uma nota por ID

```
NotaFiscal nf1 = entityManager  
                .find(NotaFiscal.class, 42);
```

```
NotaFiscal nf2 = entityManager  
                .find(NotaFiscal.class, 42);
```

```
NotaFiscal nf3 = entityManager  
                .find(NotaFiscal.class, 42);
```

Carregando uma nota por ID

```
NotaFiscal nf1 = entityManager  
                .find(NotaFiscal.class, 42);
```

```
NotaFiscal nf2 = entityManager  
                .find(NotaFiscal.class, 42);
```

```
NotaFiscal nf3 = entityManager  
                .find(NotaFiscal.class, 42);
```


Hibernate executa I select

```
NotaFiscal nf1 = entityManager.find(NotaFiscal.class, 42);
```

```
SELECT nf.*  
FROM NotaFiscal nf  
WHERE nf.id = 42
```

Hibernate executa I select

```
NotaFiscal nf1 = entityManager.find(NotaFiscal.class, 42);
```

```
SELECT nf.*  
FROM NotaFiscal nf  
WHERE nf.id = 42
```

```
NotaFiscal nf2 = entityManager.find(NotaFiscal.class, 42);
```

```
// sem SQL
```


Hibernate executa I select

```
NotaFiscal nf1 = entityManager.find(NotaFiscal.class, 42);
```

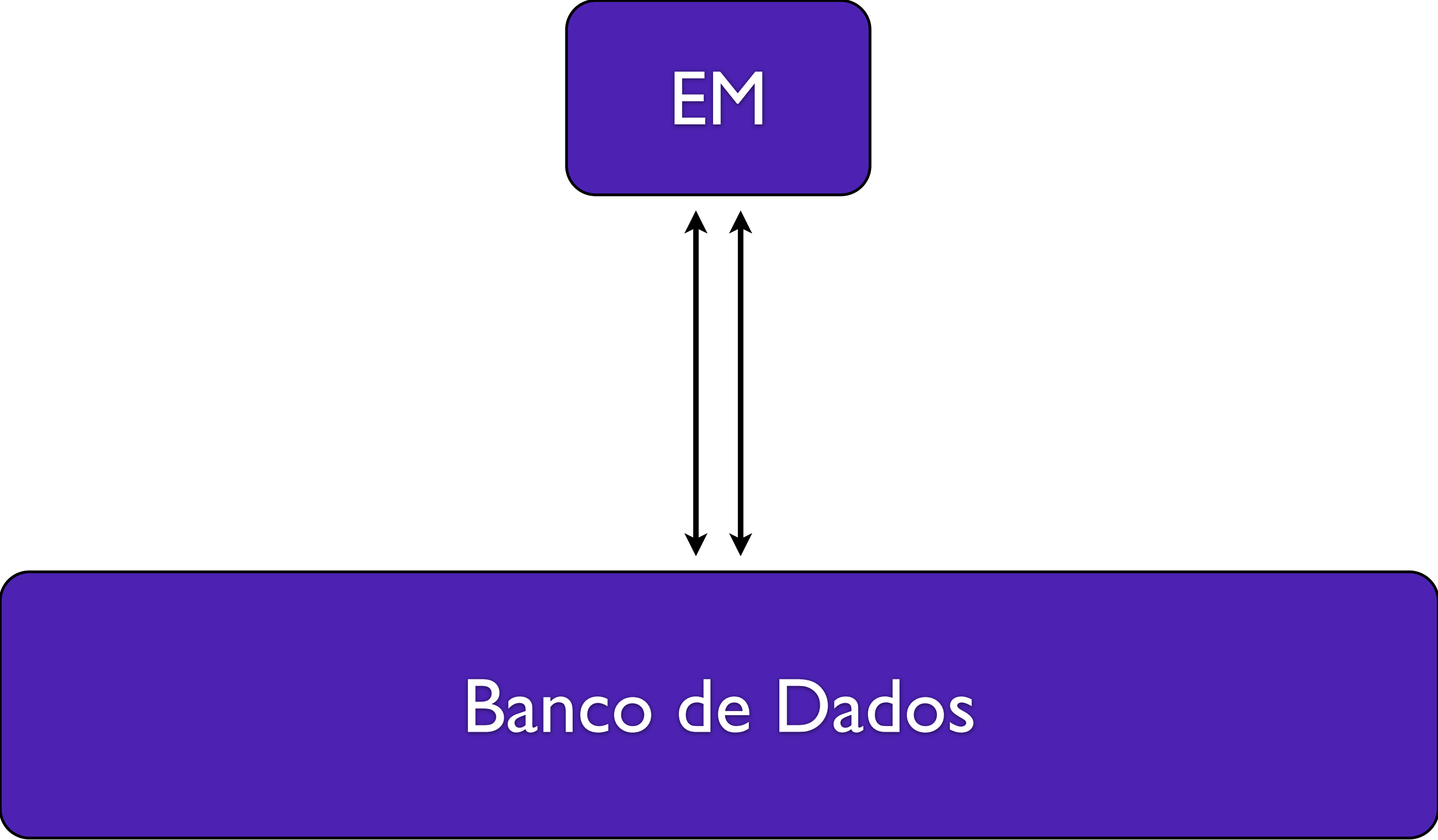
```
SELECT nf.*  
FROM NotaFiscal nf  
WHERE nf.id = 42
```

```
NotaFiscal nf2 = entityManager.find(NotaFiscal.class, 42);
```

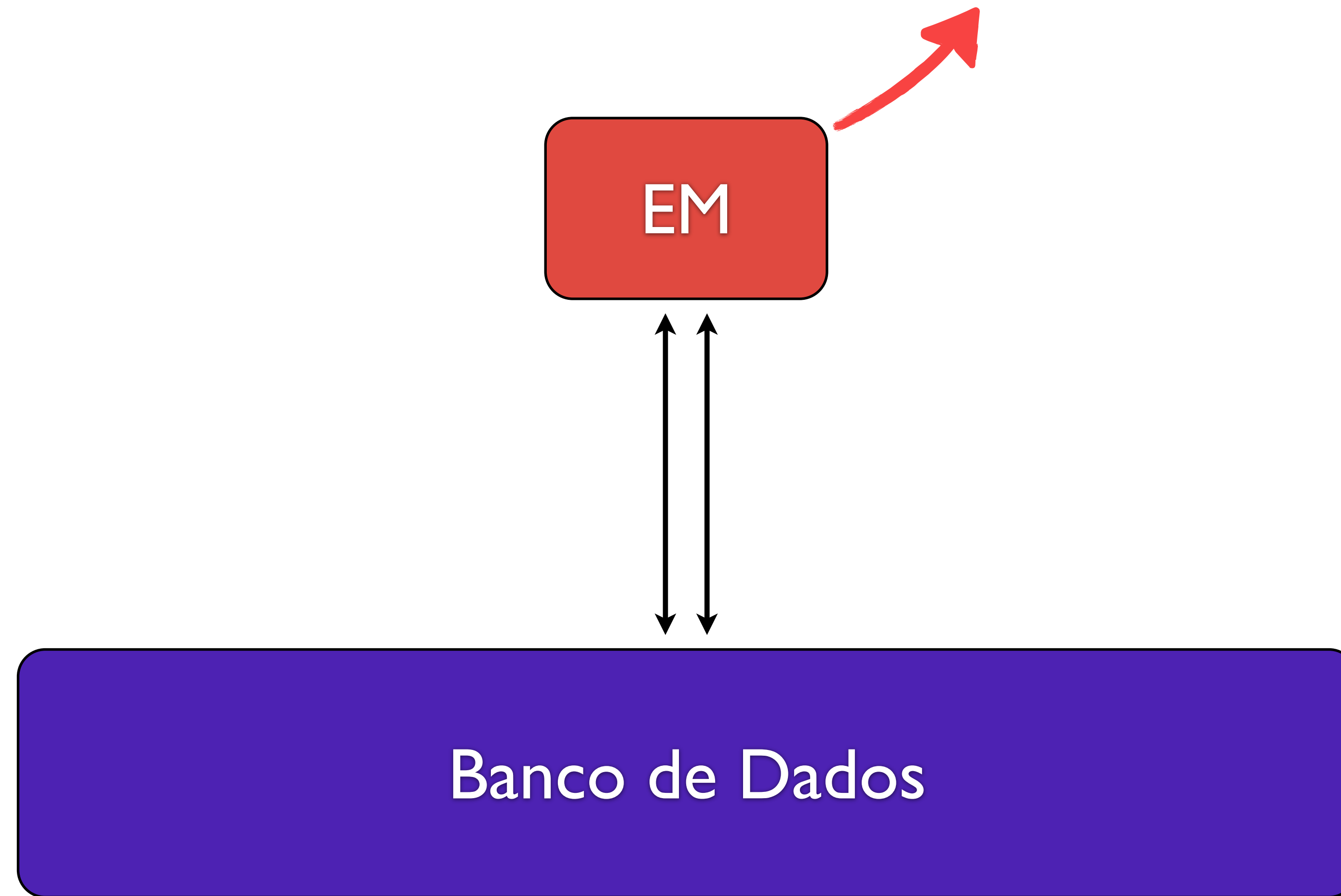
```
// sem SQL
```

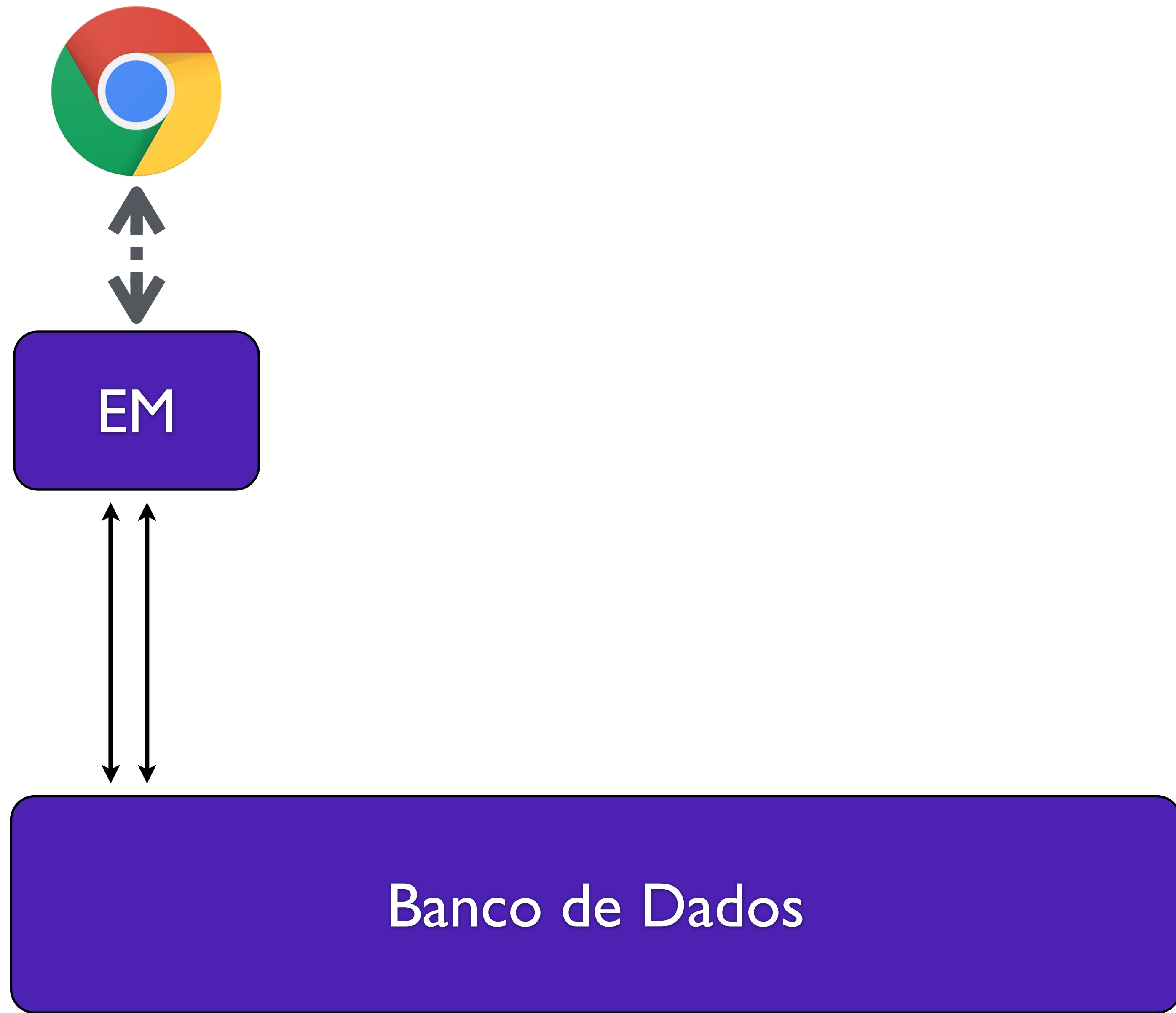
```
NotaFiscal nf3 = entityManager.find(NotaFiscal.class, 42);
```

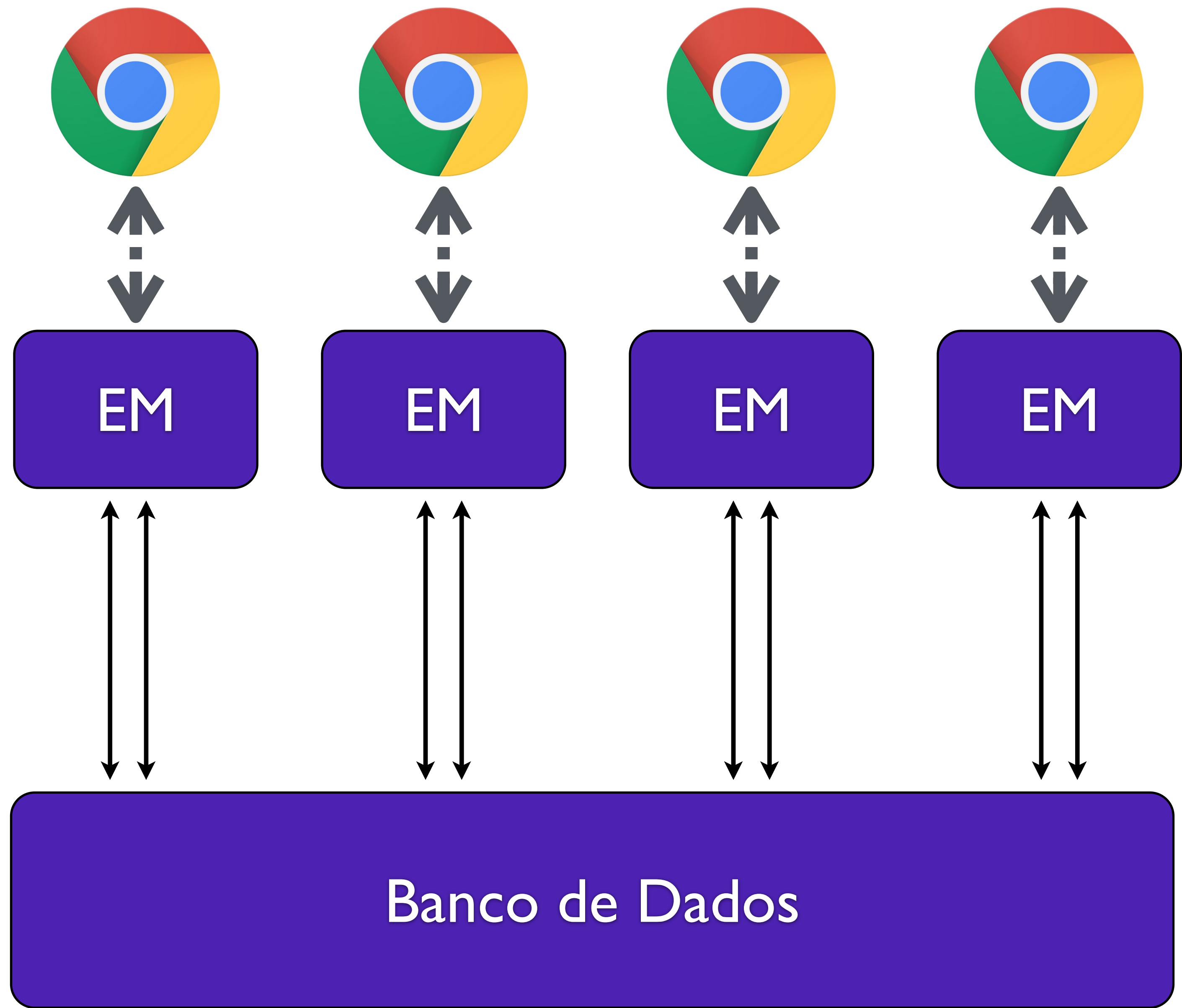
```
// sem SQL
```

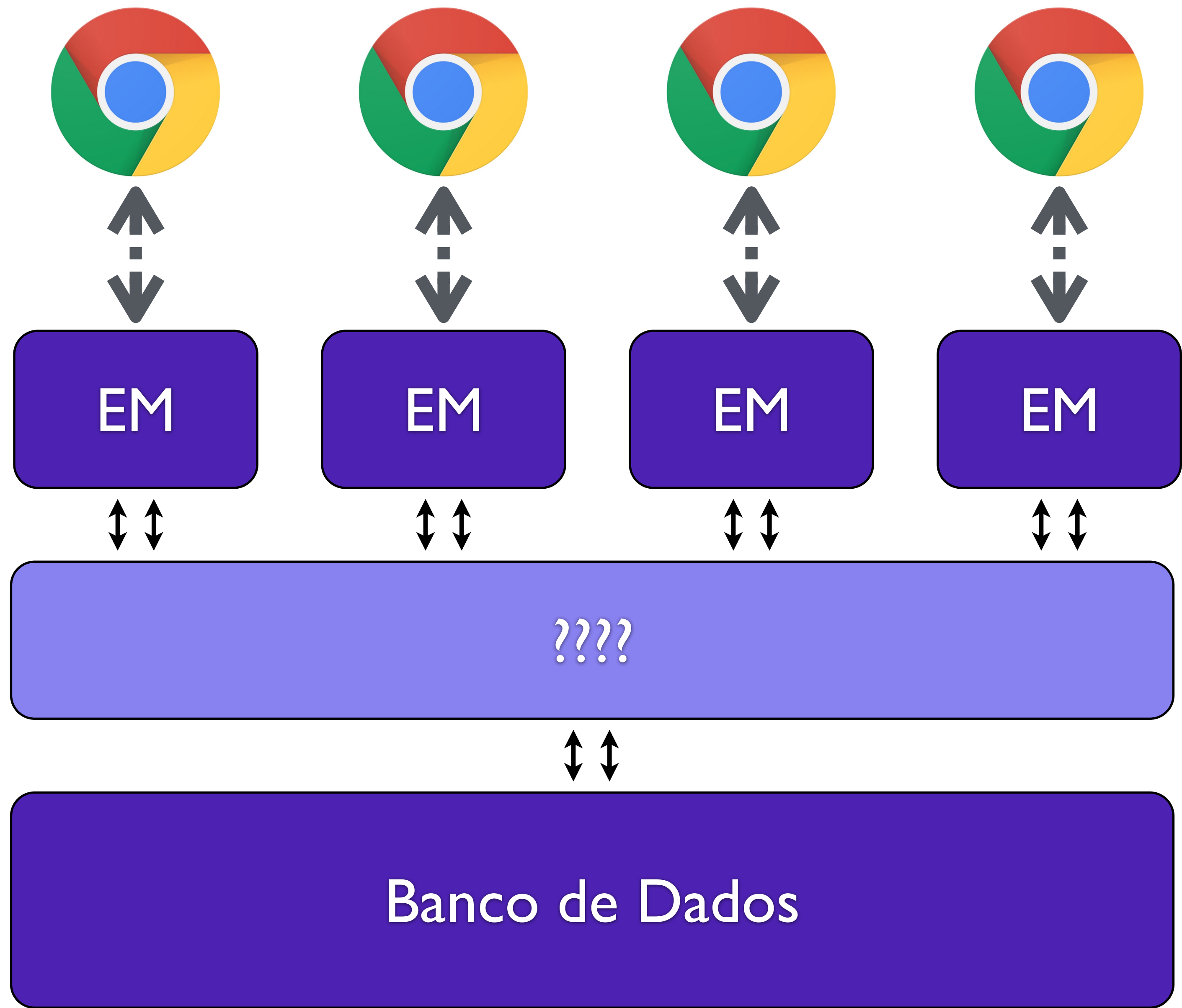


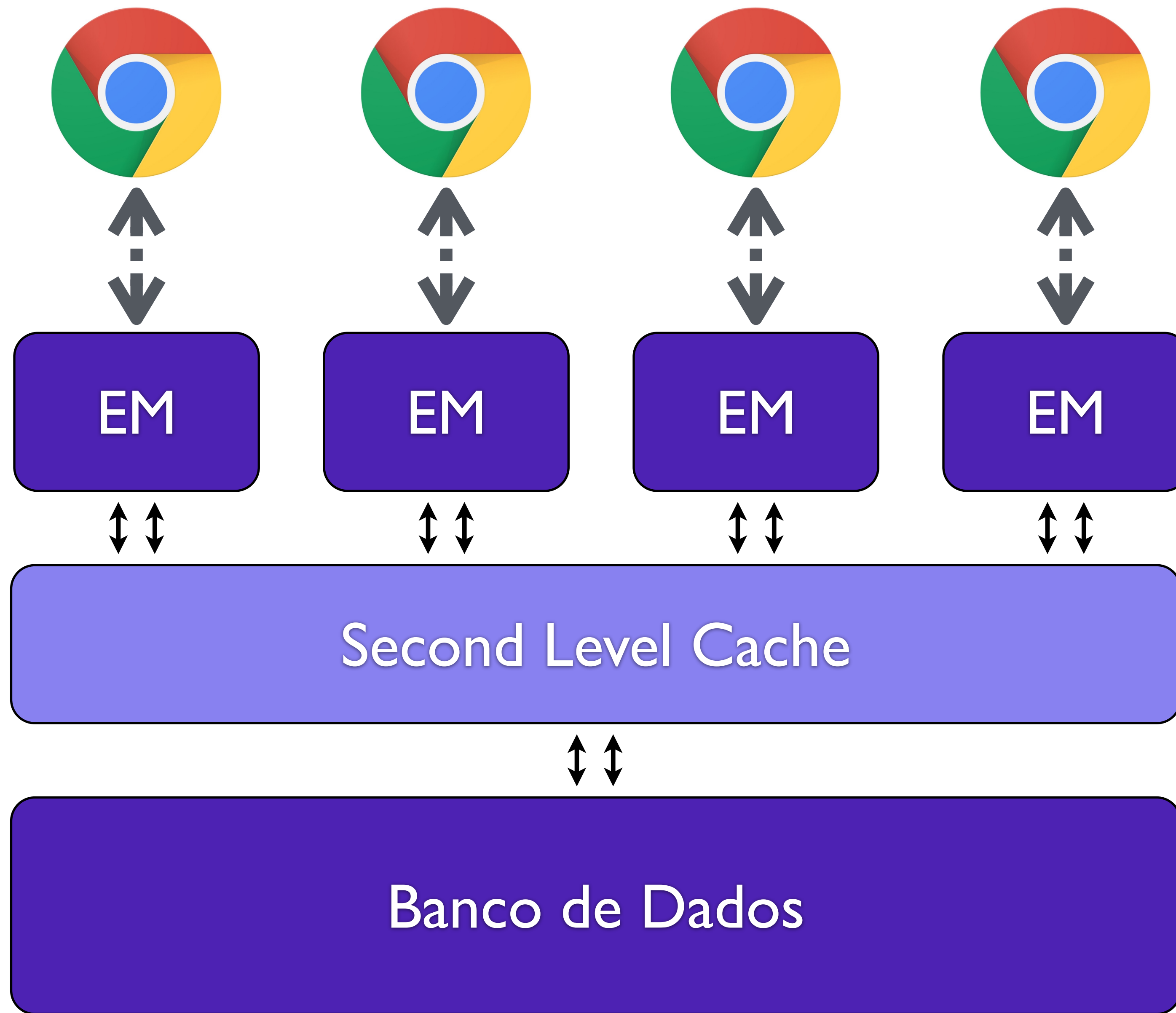
First Level Cache

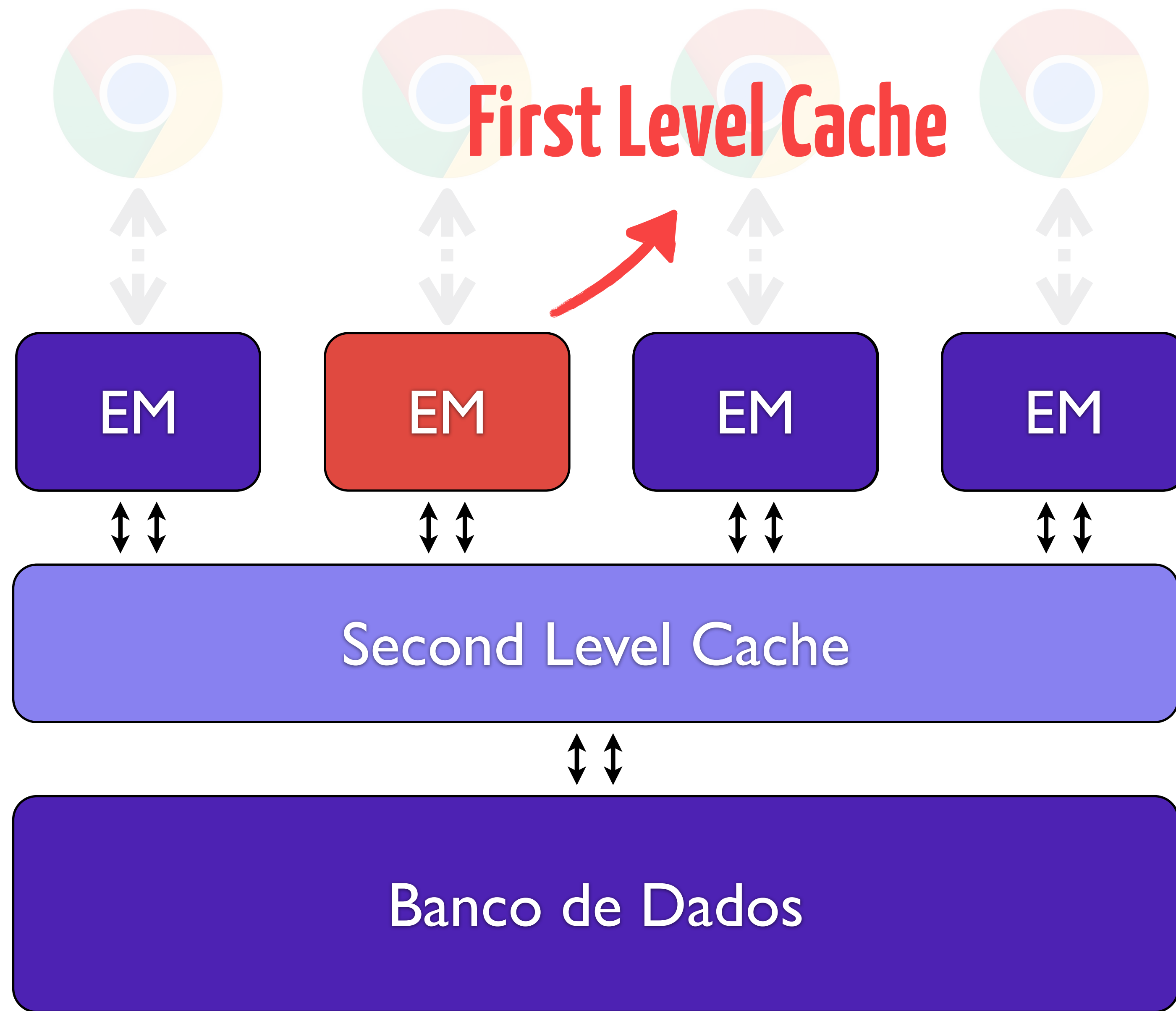


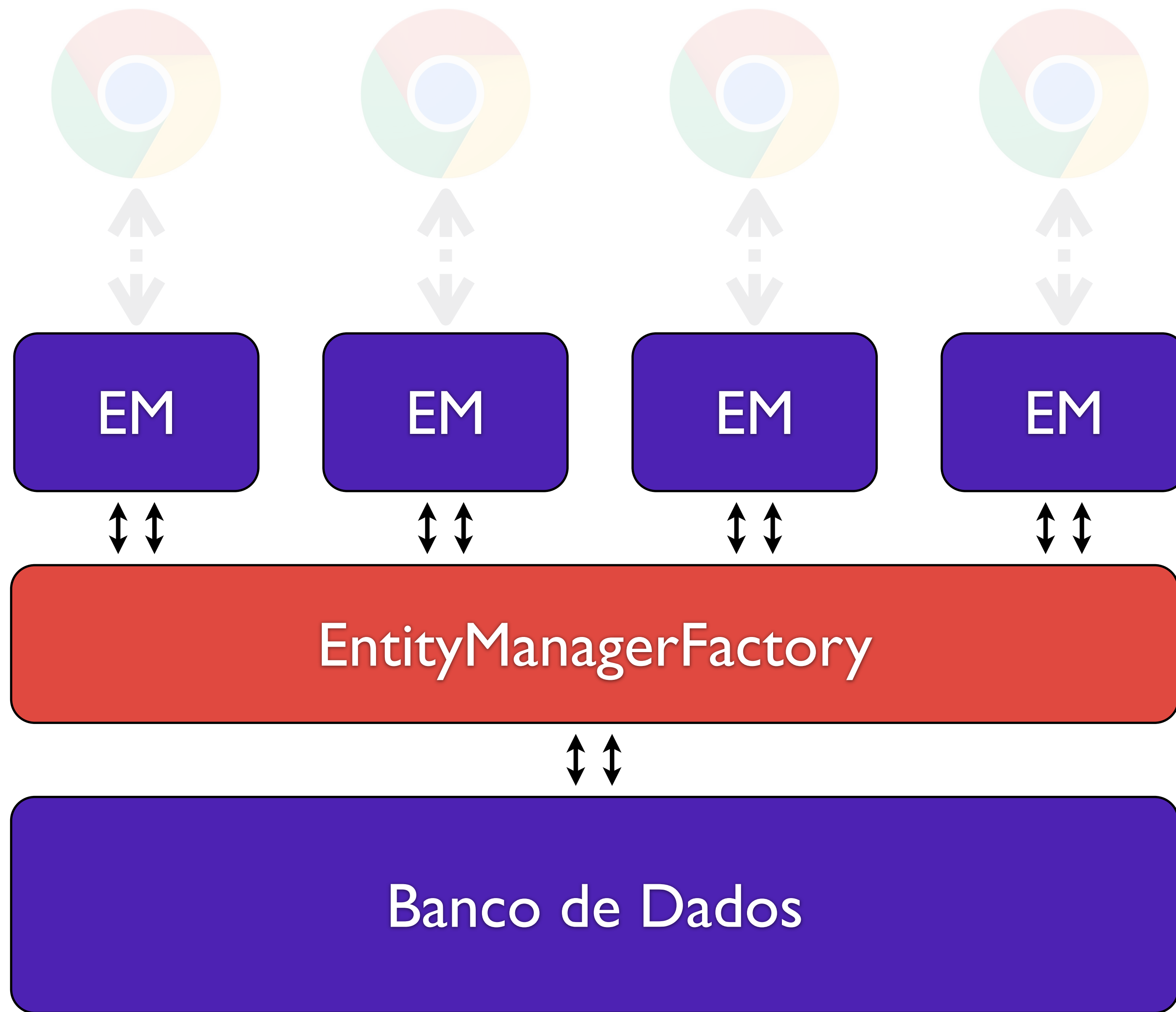












#I habilitamos o cache

```
<property  
    name="hibernate.cache.second_level_cache"  
    value="true" />
```

persistence.xml

#2 configuramos as entidades

```
@Entity
@Cache(...)
class NotaFiscal {
    ...

    @OneToMany(fetch=FetchType.EAGER)
    List<Item> itens;
}
```

#3 ...seus relacionamentos

```
@Entity  
@Cache(...)  
class NotaFiscal {  
    ...
```

```
    @Cache(...)  
    @OneToMany(fetch=FetchType.EAGER)  
    List<Item> itens;  
}
```


#4 ...e entidades auxiliares

@Entity

@Cache(...)

class Produto {

@Id

private Long **id**;

private String **nome**;

}

FINE-TUNING

Caching Strategy

```
@Cache(  
    usage=CacheConcurrencyStrategy.(???)  
)
```

Caching Strategy

```
@Cache(  
    usage=CacheConcurrencyStrategy.(???)  
)
```

READ_ONLY

NONSTRICT_READ_WRITE

READ_WRITE

Caching Strategy

```
@Cache(  
    usage=CacheConcurrencyStrategy.READ_ONLY  
)
```

READ_ONLY

Melhor performance

NONSTRICT_READ_WRITE

READ_WRITE

Caching Strategy

```
@Cache(  
    usage=CacheConcurrencyStrategy.READ_WRITE  
)
```

READ_ONLY

NONSTRICT_READ_WRITE

READ_WRITE

Modificações frequentes

Caching Strategy

```
@Cache(  
    usage=CacheConcurrencyStrategy.NONSTRICT_READ_ONLY  
)
```

READ_ONLY

NONSTRICT_READ_WRITE

READ_WRITE

Dados não críticos

persistence.xml

```
<property  
  name="hibernate.cache.second_level_cache"  
  value="true" />
```

persistence.xml

```
<property  
  name="hibernate.cache.second_level_cache"  
  value="true" />
```

```
<property  
  name="hibernate.javax.cache.provider"  
  value="org.ehcache.jsr107.EhcacheCachingProvider" />
```


persistence.xml

```
<property  
  name="hibernate.cache.second_level_cache"  
  value="true" />
```

```
<property  
  name="hibernate.javax.cache.provider"  
  value="org.ehcache.jsr107.EhcacheCachingProvider" />
```

```
<property  
  name="hibernate.cache.region.factory_class"  
  value="org.hibernate.cache...JCacheRegionFactory" />
```

persistence.xml

```
<property  
    name="hibernate.cache.second_level_cache"  
    value="true" />
```

```
<property  
    name="hibernate.javax.cache.provider"  
    value="org.ehcache.jsr107.EhcacheCachingProvider" />
```

```
<property  
    name="hibernate.cache.region.factory_class"  
    value="org.hibernate.cache...JCacheRegionFactory" />
```

```
<property  
    name="hibernate.javax.cache.uri"  
    value="ehcache.xml" />
```

ehcache.xml

```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```


ehcache.xml

<cache

name="com.deveficiente.domain.Produto"

maxElementsInMemory="100000"

eternal="false"

timeToIdleSeconds="86400" -- 24h

timeToLiveSeconds="259200" -- 72h

memoryStoreEvictionPolicy="LRU"

overflowToDisk="true"

/>

ehcache.xml

```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```

ehcache.xml

```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```


ehcache.xml

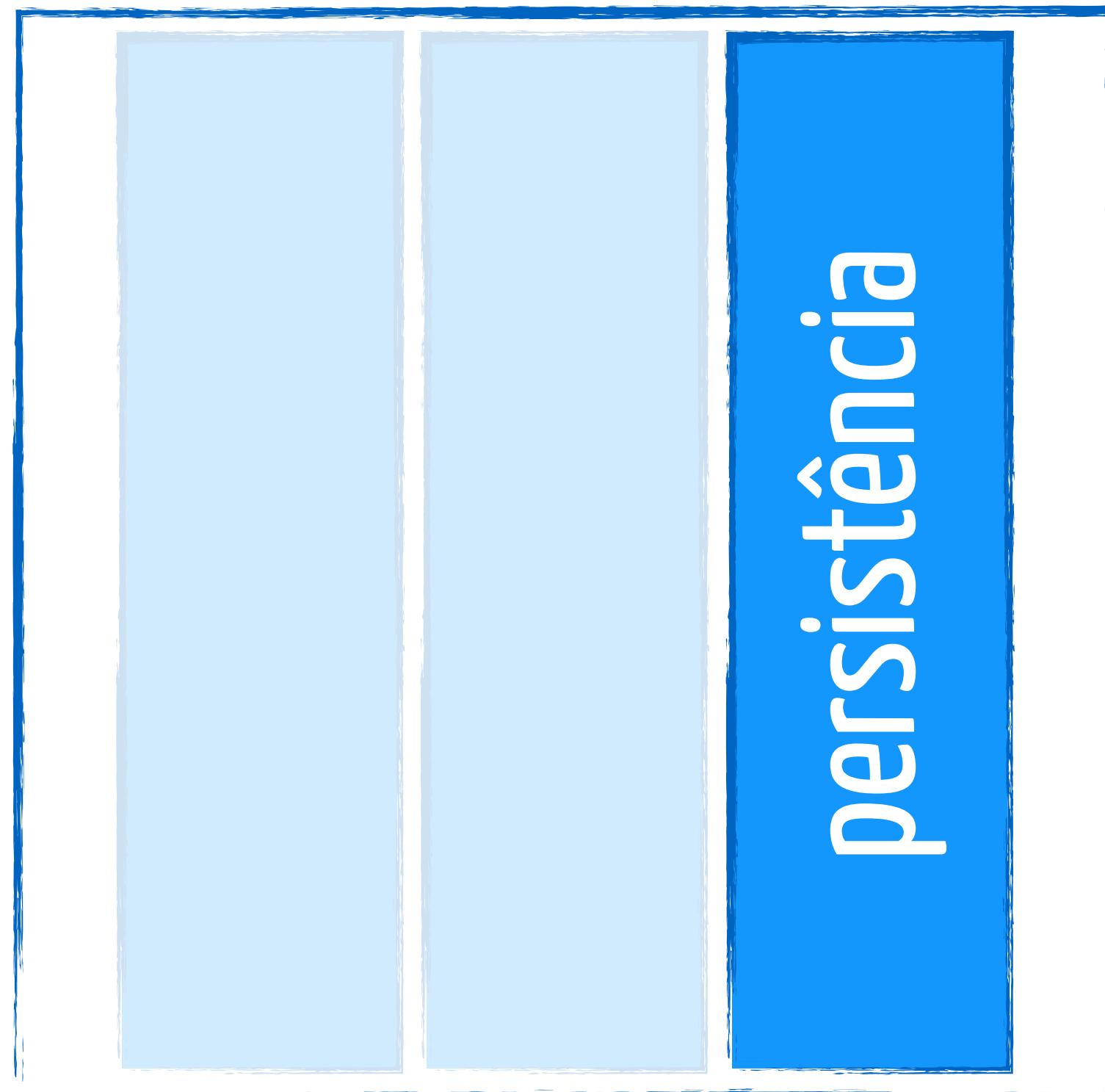
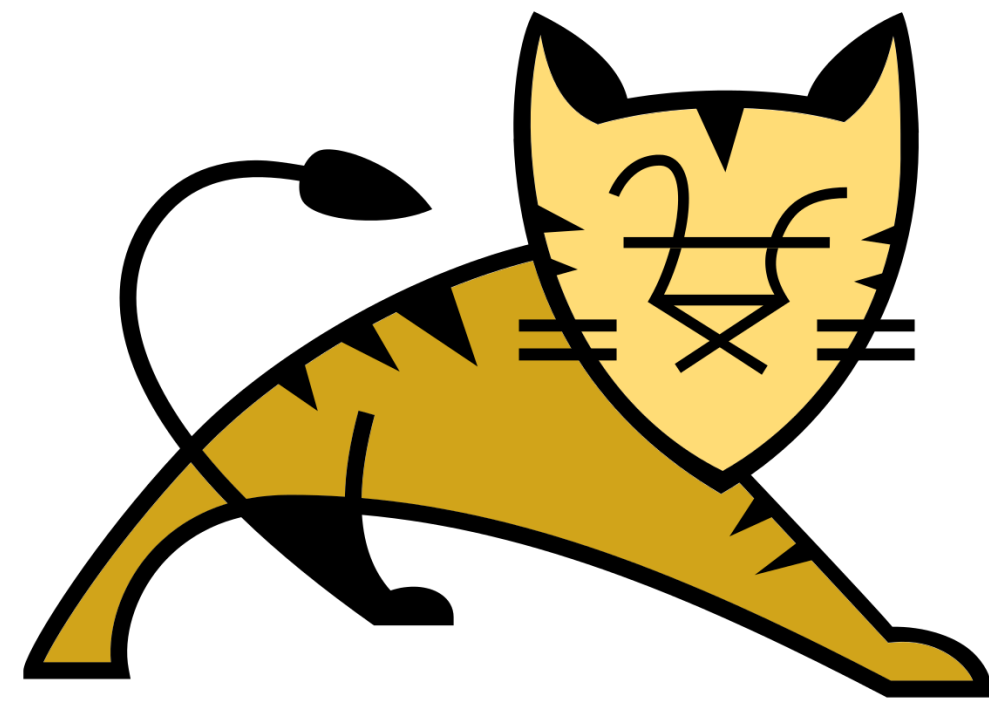
```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```

ehcache.xml

```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```


ehcache.xml

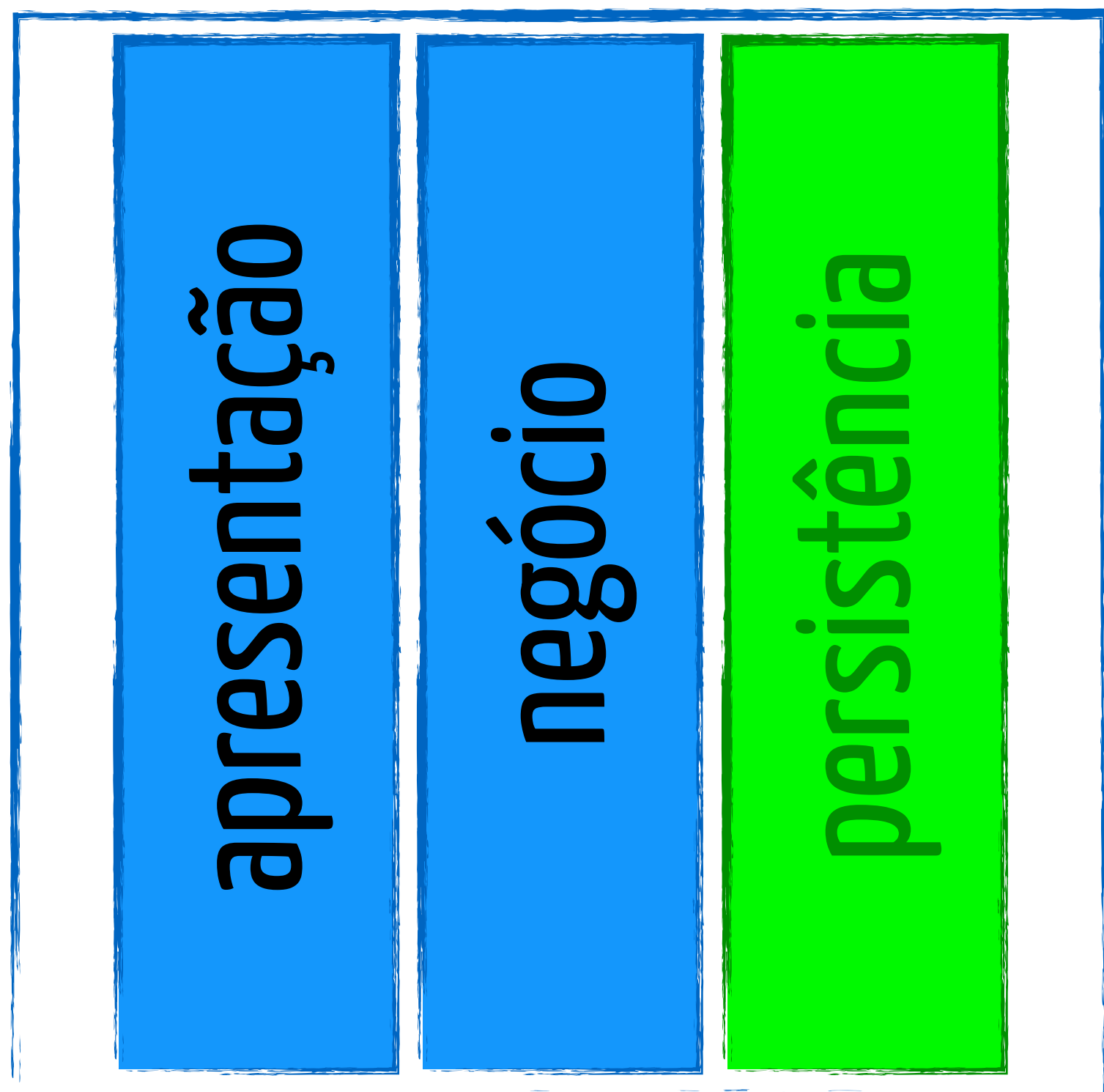
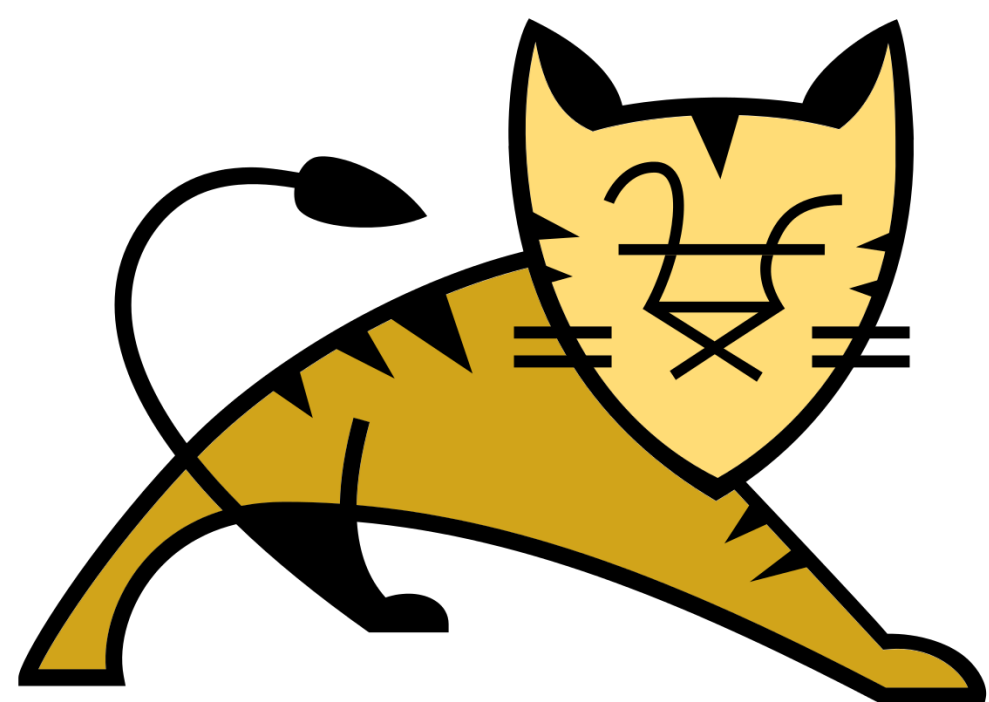
```
<cache
    name="com.deveficiente.domain.Produto"
    maxElementsInMemory="100000"
    eternal="false"
    timeToIdleSeconds="86400"    -- 24h
    timeToLiveSeconds="259200"  -- 72h
    memoryStoreEvictionPolicy="LRU"
    overflowToDisk="true"
/>
```



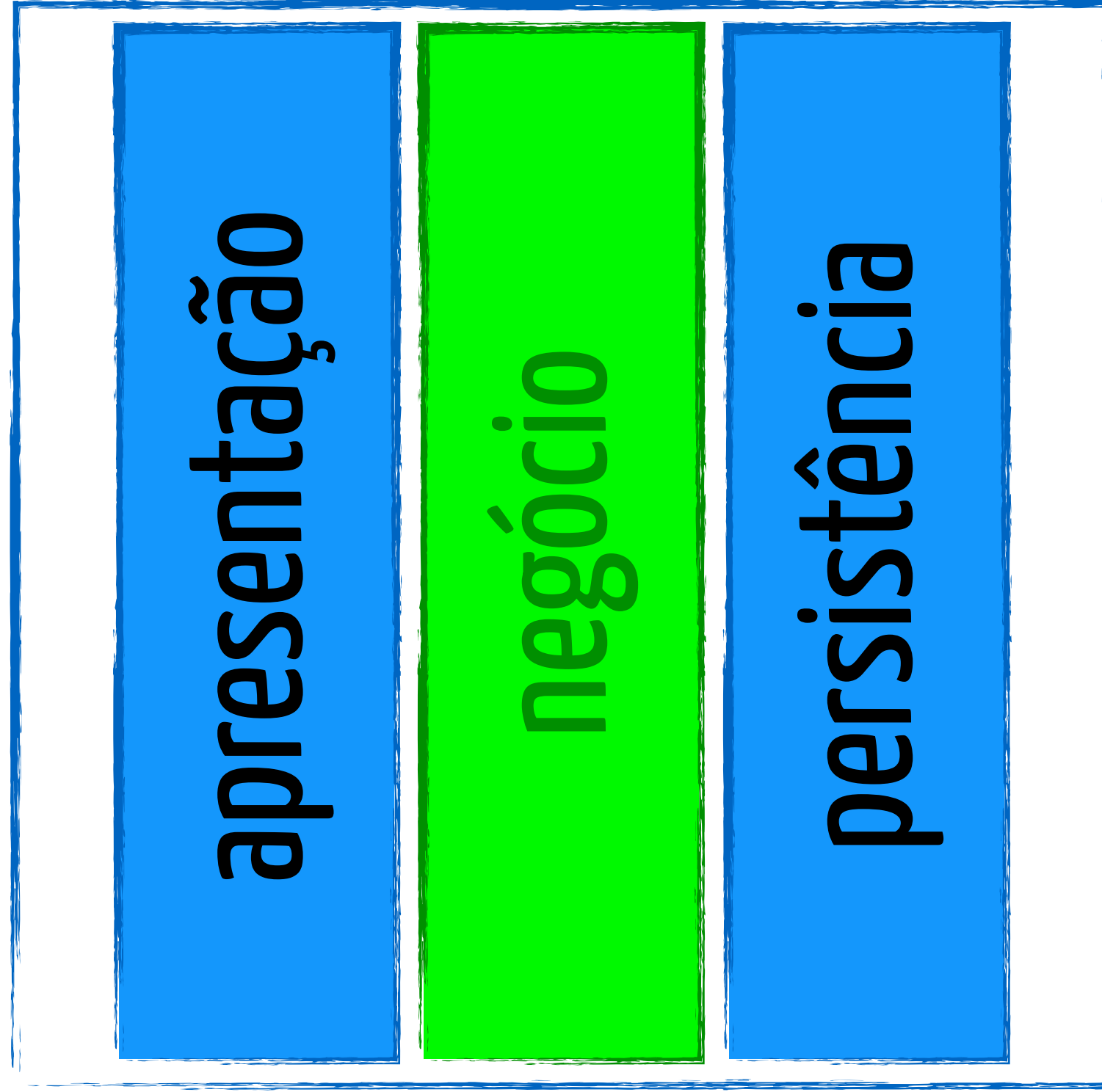
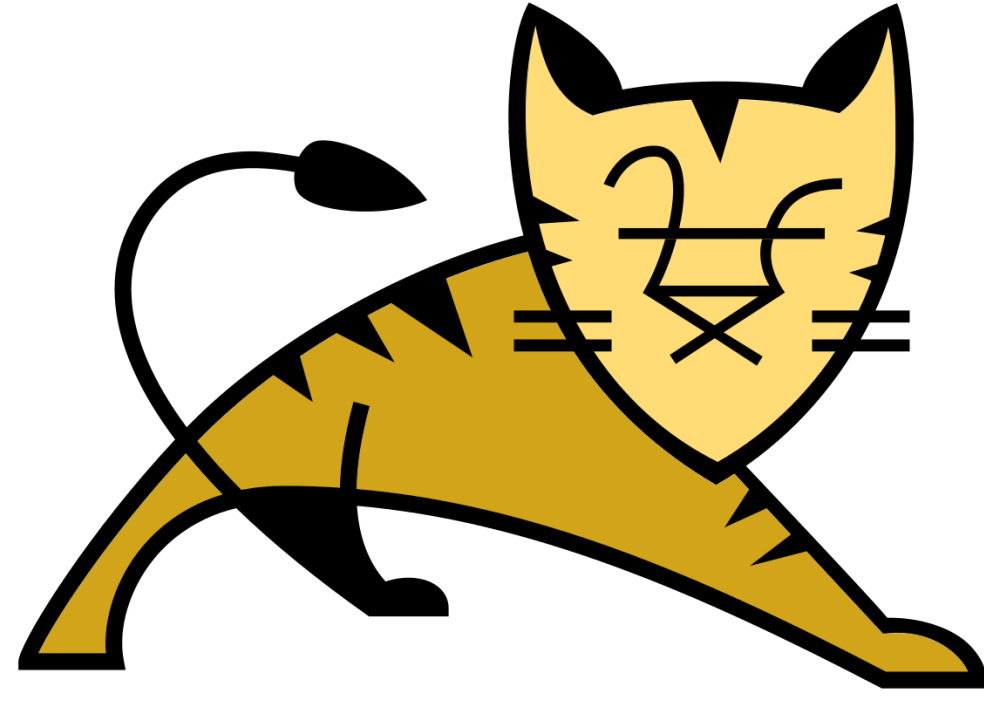
db

$$T = T_{acq} + T_{req} + T_{exec} + T_{res} + T_{idle}$$

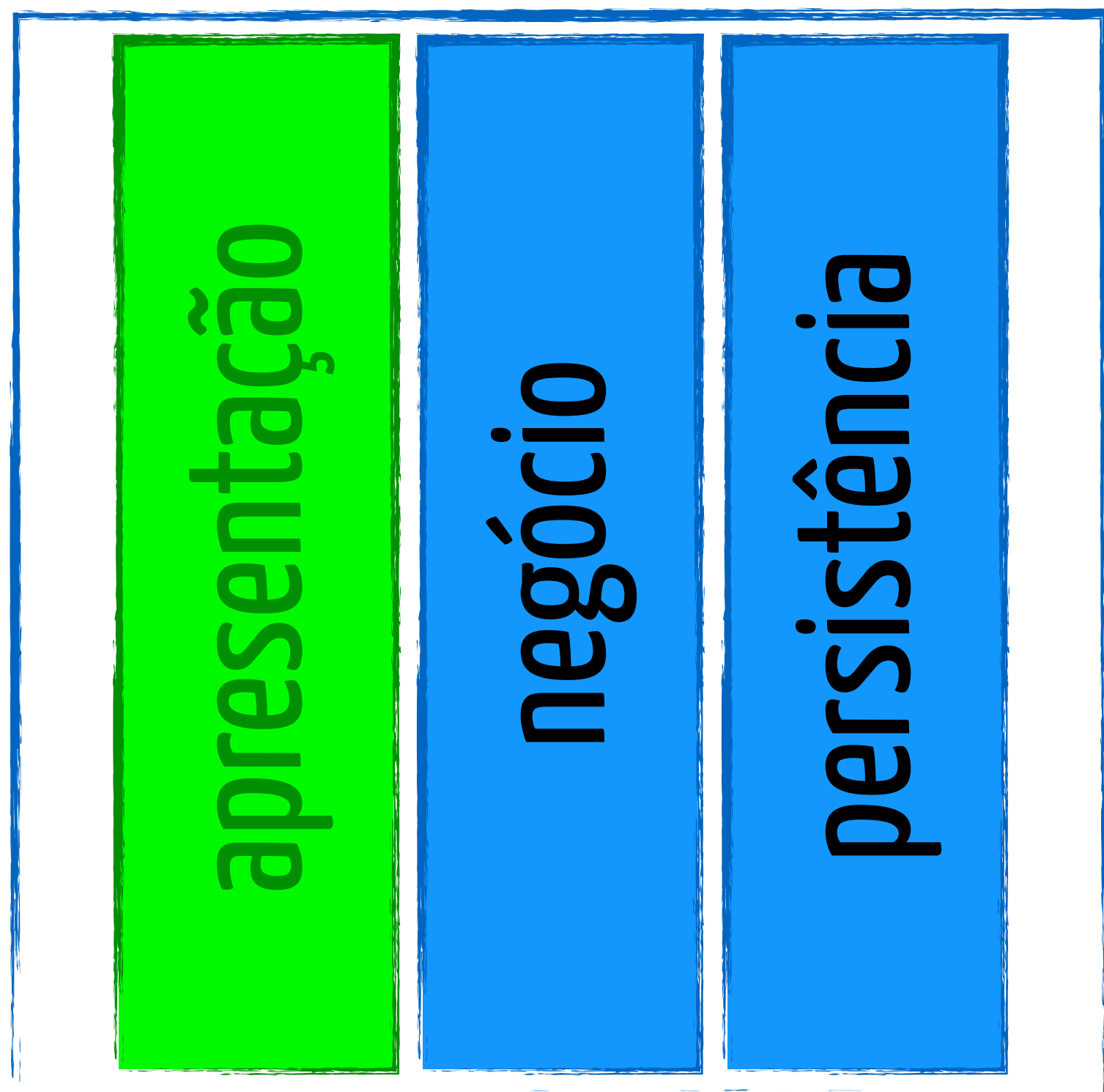
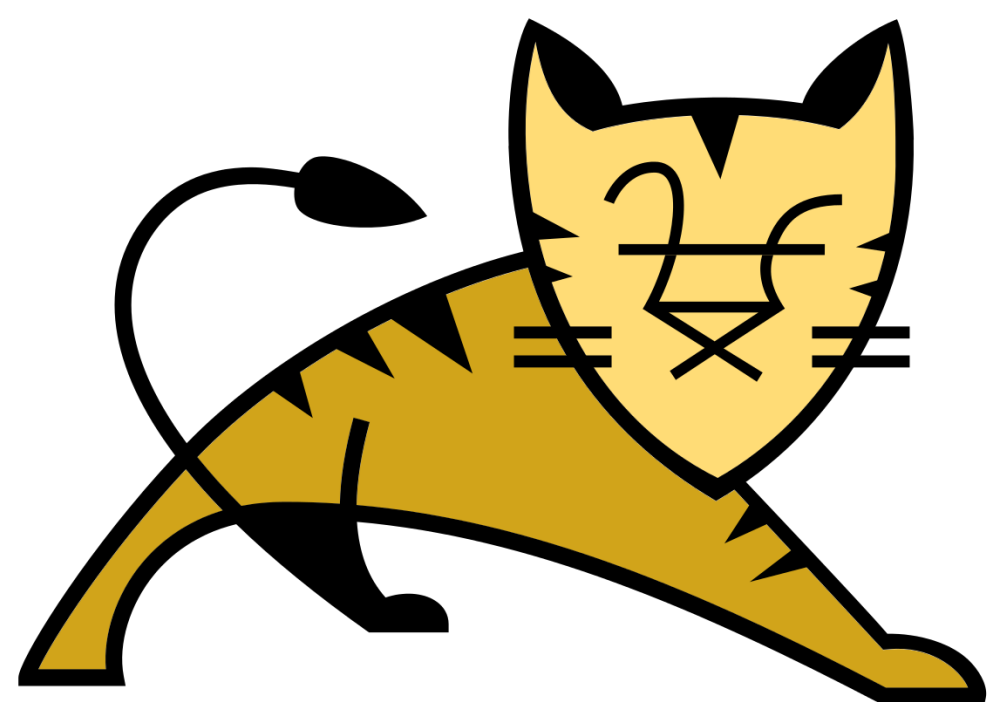
a consulta mais rápida é
aquela que **nem chega no banco
de dados**



db



db



db


```
@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow( // lança exception );

        List<Item> itens = // processa itens
        BigDecimal total = // computa total

        return // constrói e retorna payload de recibo
    }
}
```



```
@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow( // lança exception );

        List<Item> itens = // processa itens
        BigDecimal total = // computa total

        return // constrói e retorna payload de recibo
    }
}
```



```
@RestController
public class ReciboController {

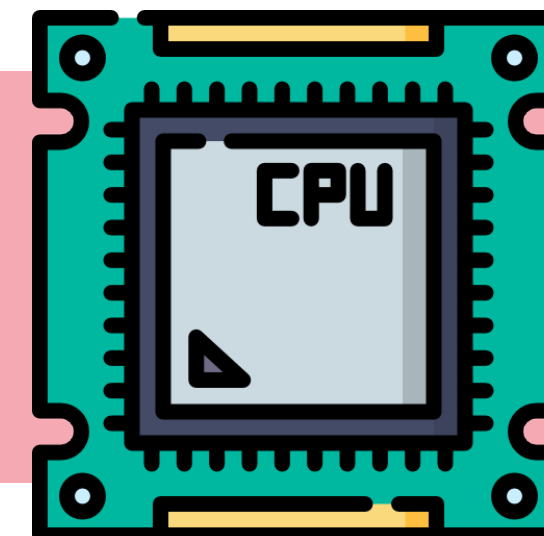
    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow( // lança exception );

        List<Item> itens = // processa itens
        BigDecimal total = // computa total

        return // constrói e retorna payload de recibo
    }
}
```



```
@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow( // lança exception );

        List<Item> itens = // processa itens
        BigDecimal total = // computa total

        return // constrói e retorna payload de recibo
    }
}
```

```
@RestController
public class ReciboController {

    private NotaFiscalRepository notaFiscalRepository;

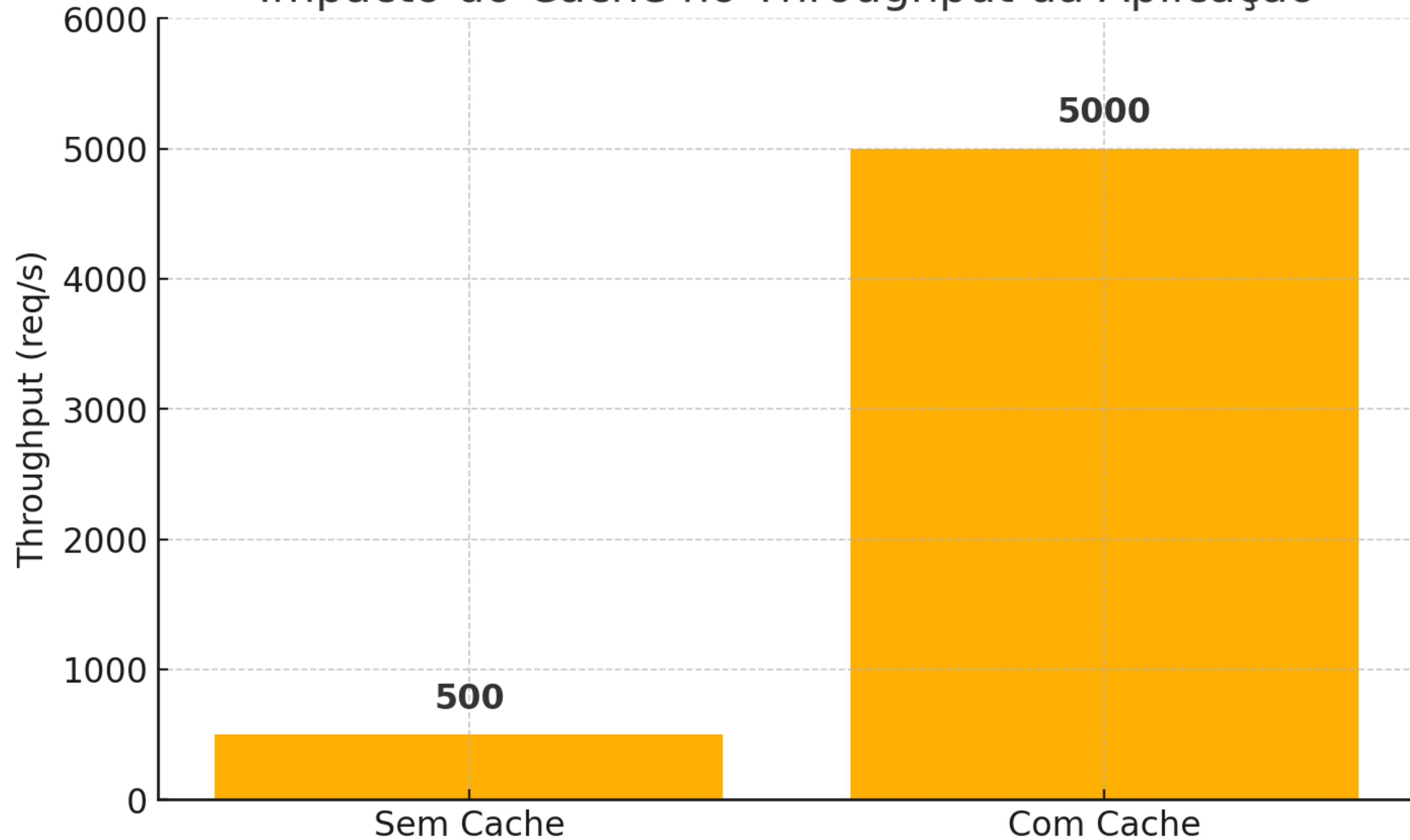
    @Cacheable("recibosCache")
    @GetMapping("/api/recibos/{notaFiscalId}")
    public ReciboResponse gerarRecibo(@PathVariable Long notaFiscalId) {

        NotaFiscal nota = notaFiscalRepository.findById(notaFiscalId)
            .orElseThrow( // lança exception );

        List<Item> itens = // processa itens
        BigDecimal total = // computa total

        return // constrói e retorna payload de recibo
    }
}
```


Impacto do Cache no Throughput da Aplicação



pagamentos.jmx (C:\Users\jackson.mota\Desktop\testes\pagamentos.jmx) - Apache JMeter (5.4.1)

Arquivo Editar Search Executar Opções Tools Ajuda

Listagem-Formas-Pagamento

Ver Árvore de Resultados

Relatório de Sumário

Grupo de Usuários

Requisição HTTP

usuariold

restauranteld

Grupo de Usuários - warmup

Relatório de Sumário

Nome: Relatório de Sumário

Comentários:

Escrever resultados para arquivo / Ler a partir do arquivo

Nome do arquivo

Procurar...

Apenas Logar/Exibir

Erros

Sucessos

Configurar

Rótulo	# Amostras	Média	Mín.	Máx.	Desvio Padrão	% de Erro	Vazão	KB/s	Sent KB/sec	Média de Bytes
Requisição HTTP	86	472	213	1049	182,49	0,00%	5,3/sec	1,16	0,86	222,0
TOTAL	86	472	213	1049	182,49	0,00%	5,3/sec	1,16	0,86	222,0

Gerenciador de Tarefas

Arquivo Opções Exibir

Processos Desempenho Histórico de aplicativos Inicializar Usuários Detalhes Serviços

Nome	Status	100% CPU	85% Memória	2% Disco	0% Rede
Vmmem		77,4%	3.234,0 MB	0,1 MB/s	0 Mbps
Host de Serviço: Serviço de Rep...		5,1%	8,6 MB	0 MB/s	0 Mbps
Host de Serviço: Serviço do Ger...		4,2%	7,2 MB	0 MB/s	0 Mbps
Host de Serviço: Serviço de Geol...		2,0%	139,7 MB	0 MB/s	0 Mbps
MinionHost		1,6%	76,9 MB	0,1 MB/s	0 Mbps
Host de Serviço: Chamada de Pr...		1,4%	18,8 MB	0 MB/s	0 Mbps
IntelliJ IDEA (9)		1,1%	3.046,2 MB	0 MB/s	0 Mbps
WMI Provider Host		1,0%	5,7 MB	0 MB/s	0 Mbps
Gerenciador de Tarefas		1,0%	24,2 MB	0,3 MB/s	0 Mbps
Gerenciador de Janelas da Área ...		0,8%	27,2 MB	0,1 MB/s	0 Mbps
Host de Serviço: Agendador de ...		0,7%	4,7 MB	0 MB/s	0 Mbps
ZscalerUpm (32 bits)		0,6%	17,9 MB	0,1 MB/s	0 Mbps
Host de Serviço: Serviços de Áre...		0,4%	52,9 MB	0 MB/s	0,3 Mbps
Windows Explorer		0,4%	36,4 MB	0,1 MB/s	0 Mbps

Menos detalhes

Finalizar tarefa

00:00:17 0 5/8

5:40 / 19:08

Incluir nome do grupo no rótulo?

Salvar Dados da Tabela

Salvar Cabeçalho da Tabel

CC

HD

#1 Java e Spring Boot - Melhoria de performance, index, query personalizada e cache

<https://www.youtube.com/watch?v=xYSPo-GfOoQ>